

Zahlen und Statisches – Teil 2

Autoboxing und String.format

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 15. Januar 2026 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

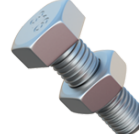
Frei nach Helmuth Karl
Bernhard von Moltke

Zahlen und Statisches

Plan für diese Woche

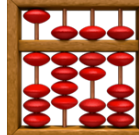
 Statische Methoden

 Statische Variablen

 Konstanten

 Schlüsselwort `final`

 Pair Programming

 Math-Methoden

 Wrapper und Autoboxing

 Zahlenformatierung

 Pair Programming

 Statische Importe

 Spaß

Zahlen und Statisches

Verbindungen herstellen

 Tauschen Sie sich mit einer Nachbar:in aus!

 Welche Arten von Methoden gibt es in der Klasse `java.lang.Math`?

 Was ist Autoboxing und was haben Wrapper damit zu tun?

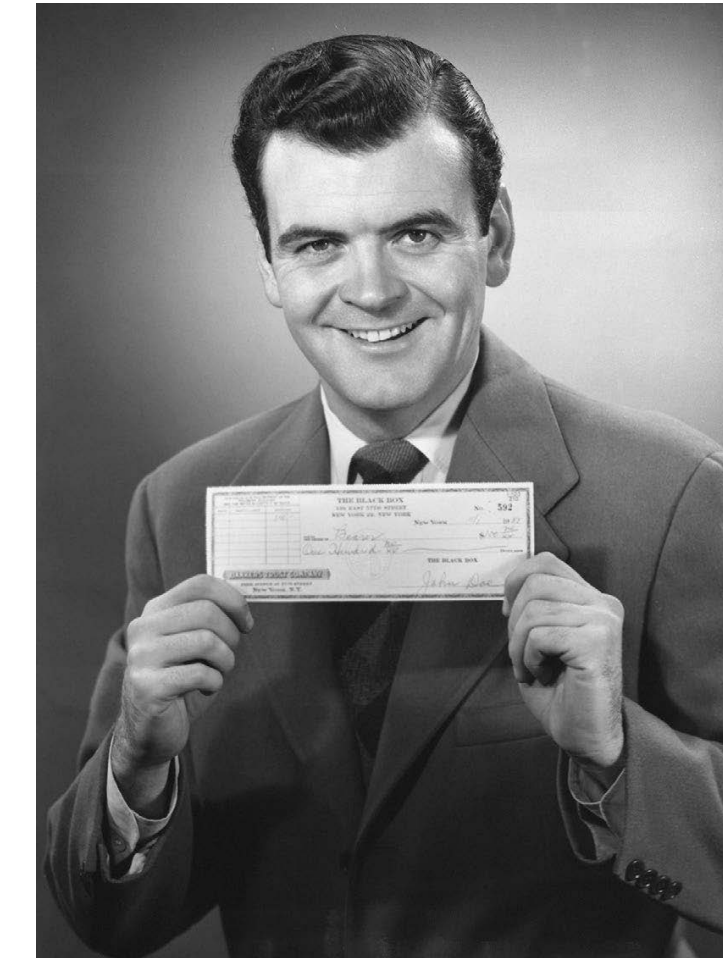
 Wie können Sie Zahlen so formatieren, dass sie gut lesbar sind?

 Was sind statische Importe und wie sollten sie verwendet werden?

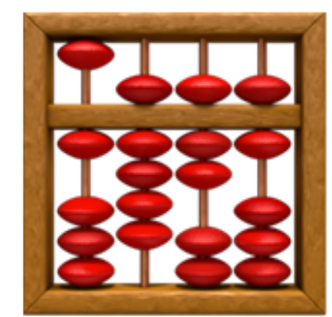
 Welche Fragen haben Sie zu diesem Kapitel?

10 Zahlen und Statisches

Zahlen, bitte!



Rechnen Sie's aus. Zur Arbeit mit Zahlen gehört deutlich mehr als nur primitive Arithmetik. Vielleicht brauchen Sie den absoluten Wert einer Zahl oder wollen eine Zahl runden oder die größere von zwei Zahlen finden. Vielleicht wollen Sie Zahlen mit genau zwei Nachkommastellen ausgeben oder große Zahlen mit Tausenderpunkten versehen, um sie lesbarer zu machen. Und was ist, wenn Sie einen String in eine Zahl umwandeln (parsen) wollen? Oder umgekehrt die Zahl in einen String konvertieren müssen? Eines Tages werden Sie eine Reihe von Zahlen in eine Collection wie eine `ArrayList` stecken wollen, die aber nur Objekte übernimmt. Glücklicherweise sind Java und die Java-API randvoll mit Möglichkeiten und Methoden für die Arbeit mit Zahlen, die Sie direkt und einfach nutzen können. Die meisten von ihnen sind **statisch** (**static**). Also lernen wir zuerst, was es für eine Variable oder Methode bedeutet, statisch zu sein, inklusive der Konstanten in Java, die auch als statische *finale* Variablen bezeichnet werden.



Math-Methoden

Field Summary

Fields

Modifier and Type	Field	Description
static final double	E	The double value that is closer than any other to <i>e</i> , the base of the natural logarithms.
static final double	PI	The double value that is closer than any other to <i>pi</i> (π), the ratio of the circumference of a circle to its diameter.
static final double	TAU	The double value that is closer than any other to <i>tau</i> (τ), the ratio of the circumference of a circle to its radius.

Method Summary

All Methods

Static Methods

Concrete Methods

Modifier and Type	Method	Description
static double	abs (double a)	Returns the absolute value of a double value.
static float	abs (float a)	Returns the absolute value of a float value.
static int	abs (int a)	Returns the absolute value of an int value.
static long	abs (long a)	Returns the absolute value of a long value.
static int	absExact (int a)	Returns the mathematical absolute value of an int value if it is exactly representable as an int, throwing <code>ArithmeticException</code> if the result overflows the positive int range.
static long	absExact (long a)	Returns the mathematical absolute value of an long value if it is exactly representable as an long, throwing <code>ArithmeticException</code> if the result overflows the positive long range.
static double	acos (double a)	Returns the arc cosine of a value; the returned angle is in the range 0.0 through <i>pi</i> .
static int	addExact (int x, int y)	Returns the sum of its arguments, throwing an exception if the result overflows an int.
static long	addExact (long x, long y)	Returns the sum of its arguments, throwing an exception if the result overflows a long.
static double	asin (double a)	Returns the arc sine of a value; the returned angle is in the range <i>-pi/2</i> through <i>pi/2</i> .

MathDemo.java

+

Math-Methoden

AI

NEW

JAVA

RUN

1

public class MathDemo {

2

public static void main(String[] args) {

3

// Commonly used Math methods

4

5

// 1. Math.abs() - Returns the absolute value of a given number

6

System.out.println("1. Absolute value of -10: " + Math.abs(-10));

7

8

// 2. Math.sqrt() - Returns the square root of a given number

9

System.out.println("2. Square root of 25: " + Math.sqrt(25));

10

11

// 3. Math.pow() - Returns the value of the first argument raised to the power of the second

12

// argument

13

System.out.println("3. 2 raised to the power of 3: " + Math.pow(2, 3));

14

15

// 4. Math.round() - Rounds a floating-point number to the nearest integer

16

System.out.println("4. Round off 3.6: " + Math.round(3.6));

17

18

// 5. Math.min() - Returns the smaller of two numbers

19

System.out.println("5. Minimum of 5 and 10: " + Math.min(5, 10));

20

21

// 6. Math.max() - Returns the larger of two numbers

22

System.out.println("6. Maximum of 5 and 10: " + Math.max(5, 10));

23

24

// Less commonly used Math methods

25

26

// 7. Math.ceil() - Rounds a number upward to the nearest integer

27

System.out.println("7. Ceiling of 3.2: " + Math.ceil(3.2));

28

29

// 8. Math.floor() - Rounds a number downward to the nearest integer

30

System.out.println("8. Floor of 3.8: " + Math.floor(3.8));

31

32

// 9. Math.log() - Returns the natural logarithm (base e) of a number

33

System.out.println("9. Natural logarithm of 10: " + Math.log(10));

34

35

// 10. Math.cos() - Returns the cosine of a number (angle in radians)

36

System.out.println("10. Cosine of PI: " + Math.cos(Math.PI));

37

38

// 11. Math.sin() - Returns the sine of a number (angle in radians)

39

System.out.println("11. Sine of PI/2: " + Math.sin(Math.PI / 2));

40

41

// 12. Math.tan() - Returns the tangent of a number (angle in radians)

42

System.out.println("12. Tangent of PI/4: " + Math.tan(Math.PI / 4));

43

}

44

}

45

STDIN

Input for the program (Optional)

Output:

1. Absolute value of -10: 10

2. Square root of 25: 5.0

3. 2 raised to the power of 3: 8.0

4. Round off 3.6: 4

5. Minimum of 5 and 10: 5

6. Maximum of 5 and 10: 10

7. Ceiling of 3.2: 4.0

8. Floor of 3.8: 3.0

9. Natural logarithm of 10: 2.302585092994046

10. Cosine of PI: -1.0

11. Sine of PI/2: 1.0

12. Tangent of PI/4: 0.9999999999999999

! Quiz

🧮 Math-Methoden

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Die Klasse Math enthält Methoden zur Durchführung grundlegender numerischer Operationen, wie beispielsweise elementare Exponential-, Logarithmus-, Quadratwurzel- und trigonometrische Funktionen.

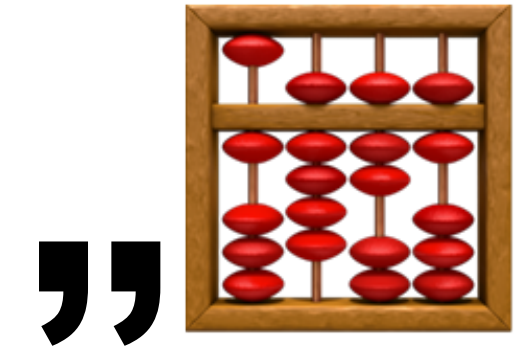
Richtig!

Sie müssen zuerst eine Instanz der Klasse Math erstellen, um sie verwenden zu können.

Nein, denn alle Methoden sind `static` – außerdem ist der Konstruktor der Klasse `private`.

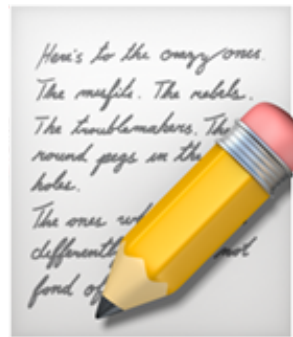


Welcher **eine** Gedanke zu



„Math-Methoden“ war für

Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren



Wrapper und Autoboxing

„Autoboxing und Unboxing ermöglichen es Entwickler:innen, saubereren Code zu schreiben, der leichter zu lesen ist.“

Frei übersetzt aus: Dev.java > Tutorials > Numbers and Strings > Autoboxing and Unboxing

„**Autoboxing** ist die automatische Konvertierung, die der Java-Compiler zwischen primitiven Typen und ihren entsprechenden **Objektwrapperklassen** durchführt. Zum Beispiel wird ein `int` in ein `Integer` konvertiert, ein `double` in ein `Double` und so weiter. Wenn die Konvertierung in die andere Richtung erfolgt, spricht man von **Unboxing**.“

Frei übersetzt aus: Dev.java > Tutorials > Numbers and Strings > Autoboxing and Unboxing

`ArrayList.add(1)`

`ArrayList.add(Integer.valueOf(1))`

Wrapper-Klassen

📦 Wrapper und Autoboxing

📦 Für jeden Elementartyp gibt es eine **Wrapper-Klasse** (vom englischen Verb „to wrap“ – „einpacken“, „umgeben“).

🌱 boolean, byte, short, char,
int, long, float, double

🌳 Boolean, Byte, Short, Character,
Integer, Long, Float, Double

🧵 Einen Wert verpacken

```
char c = 'a';  
Character cWrapped = Character.valueOf(c);
```

✂️ Einen Wert auspacken

```
char unWrapped = cWrapped.charValue();
```



„Aha! Mit einer Wrapper-Klasse kann ich einen Elementartyp wie ein Objekt behandeln.“

Gerit – aufgeweckte Student:in im 1. Semester





„Seit Java 9 sind die Konstruktoren für viele Primitive wie Integer oder Character ‚deprecated‘. Sie sollten daher nur noch die Factory-Methoden verwenden.“

„Im Buch ist von Konstruktoren die Rede, wie `new Integer(i)`. Sie erzählen uns etwas über `Integer.valueOf(i)`. Wie kommt das?“

Noel – neugierige Student:in im 1. Semester



Autoboxing und Unboxing

📦 Wrapper und Autoboxing

- Bis 2004 musste man die Elementartypen selbst ein- und auspacken.
- Mit Java 5 wurde das sogenannte Autoboxing und Unboxing eingeführt.
- Java verpackt und entpackt damit Elementartypen automatisch.



Autoboxing und Unboxing durch den Compiler

📁 Wrapper und Autoboxing

```
public static void main(String[] args) {  
    int x = 32;  
    ArrayList<Integer> list = new ArrayList<Integer>();  
    list.add(x);  
    int num = list.get(0);  
}
```

```
public static void main(String[] args) {  
    int x = 32;  
    ArrayList<Integer> list = new ArrayList<Integer>();  
    list.add(Integer.valueOf(x));  
    int num = list.get(0).intValue();  
}
```

Autounboxing

Autoboxing



Der „Java-Compiler“

**Jetzt Seiten 292
und 293
wiederholen!**



Sie haben dafür 5 Minuten Zeit.

```

1 public class TestBox {
2
3     private Integer i;
4     private int j;
5
6     public static void main(String[] args) {
7         TestBox t = new TestBox();
8         t.go();
9     }
10
11    public void go() {
12        j = i;
13        System.out.println(j);
14        System.out.println(i);
15    }
16 }
17

```

STDIN

Input for the program (Optional)

Output:

```

Exception in thread "main" java.lang.NullPointerException: Cannot invoke "java.lang.Integer.intValue()" because "this.i" is null
    at TestBox.go(TestBox.java:12)
    at TestBox.main(TestBox.java:8)

```


WrapperDemo.java

Hilfsmethoden in Wrapper-Klassen

AI

NEW

JAVA

RUN

1

public class WrapperDemo {

2

3

public static void main(String[] args) {

4

5

System.out.println("# Parsing int value represented by string");

6

7

String s = "2";

8

int i = Integer.parseInt(s);

9

System.out.println(s + " = " + i);

10

11

System.out.println("# Parsing double value represented by string");

12

13

double d = Double.parseDouble("47.11");

14

System.out.println(d);

15

16

System.out.println("# Parsing boolean value represented by string");

17

18

System.out.println("True -> " + Boolean.parseBoolean("True"));

19

System.out.println("false -> " + Boolean.parseBoolean("false"));

20

System.out.println("yes -> " + Boolean.parseBoolean("yes"));

21

22

System.out.println("# Attempting to convert string that does not have the appropriate format");

23

24

Integer.parseInt("two");

25

26

}

27

28

}

29

STDIN

Input for the program (Optional)

Output:

Parsing int value represented by string
2 = 2
Parsing double value represented by string
47.11
Parsing boolean value represented by string
True -> true
false -> false
yes -> false
Attempting to convert string that does not have the appropriate format

Exception in thread "main" java.lang.NumberFormatException: For input string: "two"
at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
at java.base/java.lang.Integer.parseInt(Integer.java:565)
at java.base/java.lang.Integer.parseInt(Integer.java:662)
at WrapperDemo.main(WrapperDemo.java:24)

```
1 public class NumberToStringDemo {
2
3     public static void main(String[] args) {
4         double d = 8.15;
5         System.out.println(d);
6
7         String concat = d + ""; // compiler converts to StringBuilder(d).append("").toString()
8         System.out.println(concat);
9
10        String toString = Double.toString(d);
11        System.out.println(toString);
12
13        String valueOf = String.valueOf(d); // calls Double.toString(d) internally
14        System.out.println(valueOf);
15    }
16 }
17
```

STDIN

Input for the program
(Optional)

Output:

8.15
8.15
8.15
8.15

! ? Quiz

📦 Wrapper und Autoboxing

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Wrapper werden verwendet, um einen Elementartyp als Objekt zu behandeln.

Richtig!

Wrapper-Klassen werden nach dem Elementartyp benannt, den sie umschließen. Der erste Buchstabe wird gemäß der Namenskonvention für Klassen groß geschrieben.

Richtig!

Die Wrapper-Klasse für char heißt Char.

Heißt Character!

Die Wrapper-Klasse für `int` heißt
Integer.

Richtig!

Für den Elementartyp `boolean` gibt es
keine Wrapperklasse.

Doch, es gibt `Boolean`!

Die Methode `parseXxx(...)` gibt immer einen `String` zurück.

... liefert den entsprechenden primitiven Wert und wirft im Fehlerfall eine Ausnahme wie `NumberFormatException`.

 Welcher **eine** Gedanke zu
„ Wrapper und Autoboxing“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Zahlenformatierung

**"PI gerundet auf vier
Nachkommastellen
beträgt 3,1416."**

`System.out.format(Locale.GERMANY, "PI gerundet auf vier Nachkommastellen beträgt %.4f.", Math.PI)`

„Die Klasse **Formatter** bietet Unterstützung für die Ausrichtung und Anpassung des Layouts, gängige Formate für numerische Daten, Zeichenketten und Datums-/Zeitangaben sowie länderspezifische Ausgaben. Es werden gängige Java-Typen wie `Byte`, `BigDecimal` und `Calendar` unterstützt.“

Frei übersetzt aus: Java® Platform, Standard Edition & Java Development Kit – Version 21 API Specification

Formatierte Konvertierung

12 34 Zahlenformatierung

- Die Klasse `java.util.Formatter` dient als Werkzeug zur formatierten Umwandlung (Konvertierung) von Daten in Strings (Text).
- Sie können damit Zahlen, Daten oder andere Informationen in einer von Ihnen definierten Weise darstellen – zum Beispiel mit bestimmten Abständen, führenden Nullen oder in einem anderen speziellen Format.
- Es ist ähnlich, wie wenn Sie einen Satz schreiben und bestimmte Wörter fett oder kursiv setzen, um sie hervorzuheben, nur dass es sich hier um Zahlen und Daten handelt.
- Sie geben die Regeln an, und `Formatter` konvertiert Ihre Daten entsprechend. Zum Beispiel: „Zahlen gerundet auf vier Nachkommastellen“ (`%.4f`).



„Das Argument als Fließkommazahl formatieren (f), auf vier Nachkommastellen runden (.4) und das lokale Dezimaltrennzeichen verwenden!“

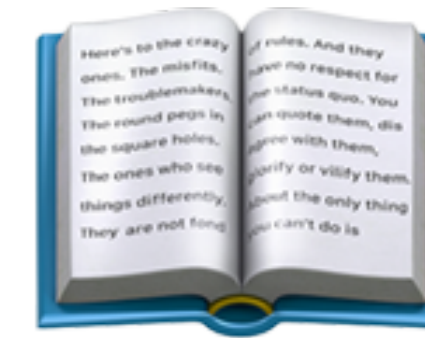
Wende diese Formatierungsanweisungen...

format(**"%.4f"**, **Math.PI**)

... auf dieses zu formatierende Argument an!

Ergebnis: zum Beispiel 3,1416 oder 3.1416,
je nach Konfiguration des Rechners.

**Jetzt Seiten 298
bis 302
wiederholen!**



Sie haben dafür 11 Minuten Zeit – inklusive Pause.

! Quiz

Zahlenformatierung

-  Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
-  Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
-  Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Sie können große Zahlen im Quellcode mit
Unterstrichen lesbarer machen:

```
Long easierToRead = 1_000_000_000;
```

Richtig!

Grundsätzlich besteht die Formatierung aus zwei Hauptbestandteilen, dem **Formatierungsstring** und den **zu formatierenden Argumenten**:

```
format("%,d", 1_000_000_000);
```

Richtig!

Das Dollarzeichen (\$) bedeutet: „Hier
Argument einfügen und gemäß diesen
Anweisungen formatieren.“

Das Prozentzeichen (%)!

% , d bedeutet: „Tausendertrennzeichen (z. B. Kommata oder Punkte) einfügen und die Zahl als dezimale Ganzzahl formatieren.“

Richtig!

`% , . 2f` bedeutet: „Die Zahl als Fließkommazahl formatieren und auf zwei Nachkommastellen runden.“

Fast: „... , dabei Tausendertrennzeichen einfügen ...“

Formatanweisungen müssen immer eine Typangabe wie %d für eine Dezimalzahl oder %f für eine Fließkommazahl enthalten. Wenn der Formatstring weitere Angaben enthält, muss die Typangabe immer am Ende stehen.

Richtig!

Variableneinfügungen im Formatstring verwenden die Reihenfolge, in der die anderen Argumente an die Methode `format()` übergeben wurden:

```
int one = 20456654;  
double two = 100567890.248907;  
String s = String.format("The rank is %,d out of %, .2f", one, two);
```

Richtig!

 Welcher **eine** Gedanke zu
„ Zahlenformatierung“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 `String.format(...)`

🤝 Pair Programming

🎯 Schreiben Sie ein Programm, das Temperaturen umrechnet und das Ergebnis im geforderten Format ausgibt.

✅ Fordern Sie zur Eingabe einer Temperatur in Celsius auf.

✅ Rechnen Sie diese in Fahrenheit um:
$$\text{fahrenheit} = (\text{celsius} * 9 / 5) + 32$$

✅ Formatieren Sie das Ergebnis mithilfe der Methode `String.format()` – oder einer anderen, eventuell besser geeigneten Methode.



String.format(...)

Pair Programming

 **Schreiben Sie ein Programm, das Temperaturen umrechnet und das Ergebnis im geforderten Format ausgibt.**

 Fordern Sie zur Eingabe einer Temperatur in Celsius auf.

 Rechnen Sie diese in Fahrenheit um:
`fahrenheit =
 (celsius * 9 / 5) + 32`

 Formatieren Sie das Ergebnis mithilfe der Methode `String.format()` – oder einer anderen, eventuell besser geeigneten Methode.

 Runden Sie die Ausgabe dabei auf zwei Dezimalstellen, zum Beispiel:
`-5.51 °C is equal to 22.08 °F`

 Bonus: Stellen Sie sicher, dass die Ausgabe bei positiven Werten ein +-Zeichen enthält.
Beispiel:
`-5.51 °C is equal to +22.08 °F`

 Am Ende sollte das Ergebnis in leicht lesbarer Form auf der Konsole angezeigt werden.


```
1 import java.util.Scanner;
2 import java.util.Locale;
3
4 public class TemperatureConverter {
5     public static void main(String[] args) {
6         // Make sure that the input and output respect the given region.
7         Locale.setDefault(Locale.US);
8
9         Scanner scanner = new Scanner(System.in);
10
11         System.out.print("Enter the temperature in degrees Celsius: ");
12         double celsius = scanner.nextDouble();
13         System.out.println();
14
15         double fahrenheit = (celsius * 9 / 5) + 32;
16
17         String format = "%.2f °C is equal to %.2f °F%n";
18         String formatPlus = "%+.2f °C is equal to %+.2f °F%n";
19
20         // This is the most common method of printing directly to the console
21         // and is often used for console output (System.out or System.err)
22         // because the name "print" implies output. It is short and concise
23         // ("syntactic sugar").
24         System.out.println("Using System.out.printf(...):");
25
26         System.out.printf(format, celsius, fahrenheit);
27         System.out.printf(formatPlus, celsius, fahrenheit);
28
29         // This is often used to explicitly emphasize that formatting is
30         // taking place, as opposed to simply printing.
31         System.out.println("Using System.out.format(...):");
32
33         System.out.format(format, celsius, fahrenheit);
34         System.out.format(formatPlus, celsius, fahrenheit);
35
36         // Unlike printf()/format(), which write directly to the stream,
37         // String.format() first creates a new string object in memory (on
38         // the heap). Use this method when you need the formatted text for
39         // storage, logging, or GUI display, rather than printing it
40         // immediately.
41         System.out.println("Using String.format(...):");
42
43         System.out.print(String.format(format, celsius, fahrenheit));
44         System.out.print(String.format(formatPlus, celsius, fahrenheit));
45
46         scanner.close();
47     }
48 }
49
```

STDIN

-5.51

Output:

Enter the temperature in degrees Celsius:

Using System.out.printf(...):

-5.51 °C is equal to 22.08 °F

-5.51 °C is equal to +22.08 °F

Using System.out.format(...):

-5.51 °C is equal to 22.08 °F

-5.51 °C is equal to +22.08 °F

Using String.format(...):

-5.51 °C is equal to 22.08 °F

-5.51 °C is equal to +22.08 °F

 Welcher **eine** Gedanke zu
„ `String.format(...)`“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren



Statische Importe


```
1 // The static import construct allows unqualified access to static members.
2 // Either individually:
3 import static java.lang.System.out;
4 // Or en masse:
5 import static java.lang.Math.*;
6
7 public class StaticImportsDemo {
8
9     public static void main(String[] args) {
10         // Once the static members have been imported, they can be used without qualification:
11         out.println("Absolute value of -10: " + abs(-10));
12         out.println("Square root of 25: " + sqrt(25));
13
14         // So when should you use static import? *Very sparingly!* ...
15         // -> https://docs.oracle.com/javase/8/docs/technotes/guides/language/static-import.html
16     }
17 }
18
```

STDIN

Input for the program (Optional)

Output:

Absolute value of -10: 10
Square root of 25: 5.0

„Verwenden Sie statische Importe sparsam. Verwenden Sie sie nur für häufig verwendete statische Member einiger weniger Klassen. Eine übermäßige Verwendung kann zu unübersichtlichem und schwer wartbaren Code führen. Importieren Sie bestimmte Member einzeln, um Klarheit und Lesbarkeit zu gewährleisten.“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt



! ? Quiz

Statische Importe

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Statische Importe können die Lesbarkeit
Ihres Codes nur verbessern. Versuchen Sie
daher, sie so oft wie möglich zu verwenden!

... erheblich einschränken. Verwenden Sie sie daher sparsam und überlegt!

In der statischen Importdeklaration können
Platzhalter (.*) verwendet werden.

Richtig!

Die Verwendung von statischen Importen entfernt die Information, aus welcher Klasse der Import stammt. Es wird daher empfohlen, statische Importe nur dann zu verwenden, wenn die statische Methode oder Variable auch ohne das Klassennamenspräfix hinreichend selbsterklärend ist.

Richtig!

Lesen Sie Ihren Code nach der Verwendung eines statischen Imports auf jeden Fall noch einmal durch und fragen Sie sich: „Verstehe ich das auch noch in einem halben Jahr?“

Richtig!

 Welcher **eine** Gedanke zu
„ Statische Importe“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Zahlen und Statisches

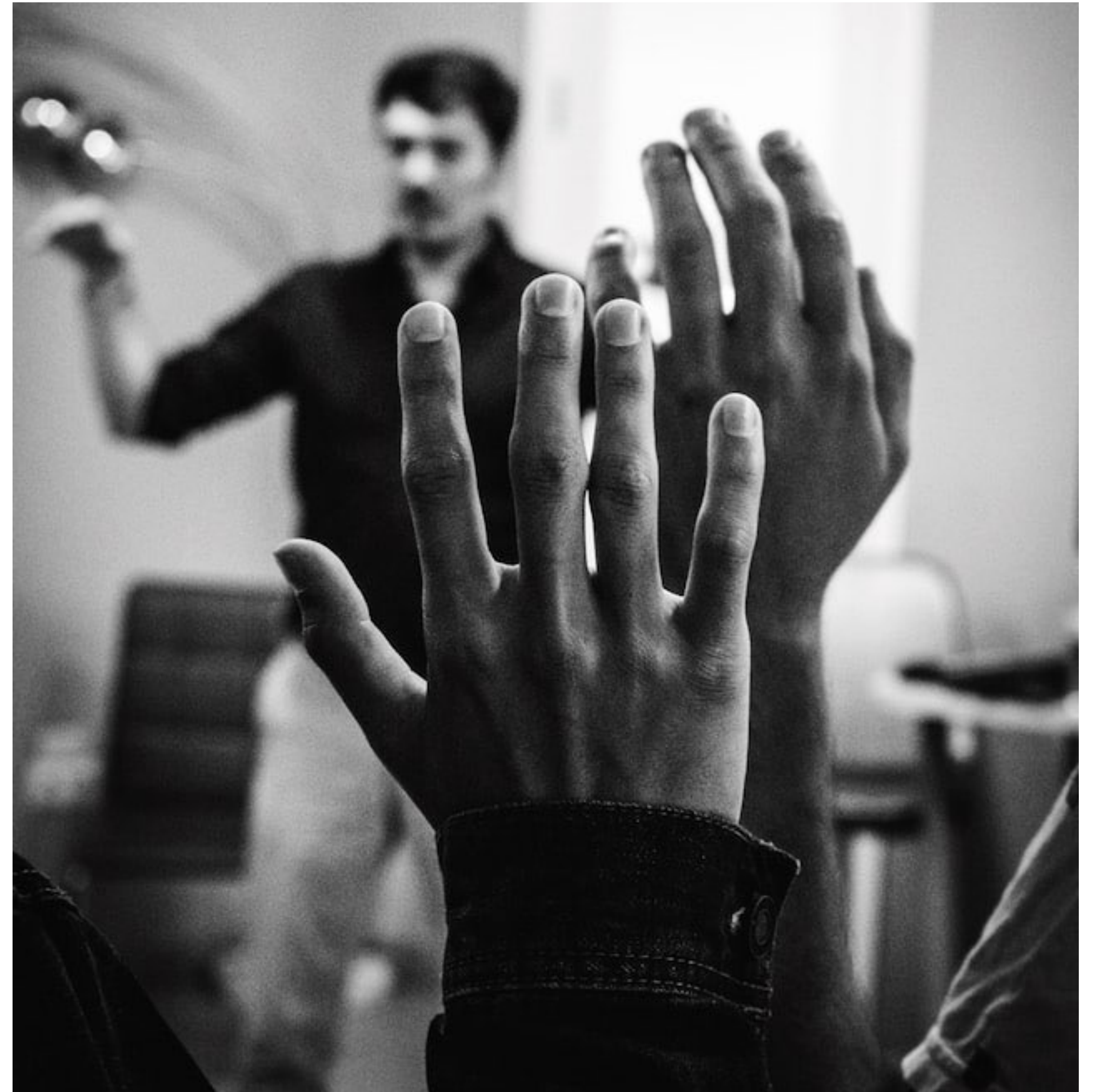
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Zahlen und Statisches?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Zahlen und Statisches



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

